

UNIVERSITE D'ANGERS

Département Informatique

RAPPORT DE SOUTENANCE DE PROJET

Année universitaire 2025 – 2026

Application Web de
Constitution de Poules
Sportives

Réalisé par :

Maroua BEHIH
Touré BOUKARI

Tuteur :

M. MARC LEGEAY

Date de soutenance :

JJ mois 2026

Remerciements

Nous tenons tout d'abord à remercier ...

Table des matières

Remerciements	1
1 Introduction	4
1.1 Contexte général	4
1.2 Problématique	4
1.3 Objectifs du projet	5
2 Organisation du stage	6
2.1 Diagramme de Gantt	6
2.2 Organisation et répartition du travail	6
3 Modélisation des données et fichiers d'entrée	7
3.1 Modélisation des données	7
3.2 Données en entrée de l'application	9
3.2.1 Fichier des clubs	9
3.2.2 Fichier des équipes	10
3.2.3 Fichier des poules	11
3.2.4 Fichier des souhaits des clubs	12
4 Conception des algorithmes de répartition des équipes	13
4.1 Étapes préliminaires communes	13
4.1.1 Vérifications préalables	13
4.1.2 Définition des capacités des poules	14
4.1.3 Initialisation des poules	15
4.2 Algorithme de répartition par distance	16
4.2.1 Sélection des graines — K-Means++	16
4.2.2 Affectation des équipes restantes	18
4.2.3 Stratégie de sauvetage	20
4.2.4 Rééquilibrage des distances moyennes	20
4.3 Algorithme de répartition par niveau	23
4.3.1 Prérequis	23
4.3.2 Calcul des quotas par statut et par poule	23

4.3.3	Distribution des équipes	24
4.3.4	Optimisation géographique	24
5	Numérotation des équipes	25
5.1	Prérequis — Uniformité des tailles de poules	25
5.2	Notion de numéro opposé	25
5.3	Algorithme de numérotation	25
5.4	Gestion des conflits	27
6	Choix des technologies utilisées	28
6.1	Choix du langage : JavaScript natif	28
6.2	Choix de la cartographie : Leaflet.js	28
6.3	Mise en service via un serveur HTTP local	29
6.3.1	Mise en place	29
6.4	Architecture générale de l'application	30
7	Implémentation et défis techniques	31
7.1	Configuration des paramètres de génération	31
7.2	Lecture et parsing du fichier CSV contenant les informations sur les clubs	33
7.3	Lecture et parsing du fichier CSV contenant les informations sur les équipes à chaque niveau	34
7.4	Génération des poules	35
7.4.1	Mode distance (<code>calcul-poules.js</code>)	35
7.4.2	Mode niveau (<code>calcul-poules-niveau.js</code>)	37
7.5	Visualisation cartographique avec Leaflet	37
7.6	Visualisation tabulaire	38
7.6.1	Possibilité d'échange manuel des équipes	39
7.7	Persistance des données avec LocalStorage	40
7.8	Affichage des résultats et export	41
7.8.1	Récapitulatif des poules	41
7.8.2	Résultat de la numérotation	42
8	Conclusion et perspectives	43
8.1	Bilan du projet	43
8.2	Perspectives d'évolution	43
	Bibliographie et Webographie	44

Chapitre 1

Introduction

1.1 Contexte général

Dans le cadre des compétitions sportives amateur et semi-professionnelles, les fédérations sportives délèguent l'organisation des championnats locaux à des structures régionales et départementales. Ces championnats adoptent fréquemment un format par **poules**. Les équipes participantes sont réparties en sous-groupes homogènes au sein desquels chaque équipe rencontre toutes les autres au moins une fois. Ce format permet de gérer un nombre élevé d'équipes tout en garantissant un nombre de matchs suffisant pour établir un classement fiable.

La constitution de ces poules doit concilier plusieurs exigences telles que le fait de regrouper des équipes de niveau comparable, de minimiser les distances de déplacement entre clubs, et de respecter des règles fédérales telles que l'interdiction de placer deux équipes d'un même club dans la même poule.

Une fois les poules constituées, chaque équipe doit recevoir un **numéro** compris entre 1 et T , où T désigne la taille commune de toutes les poules. Ce numéro est utilisé par l'organisateur pour construire le calendrier des rencontres en tenant compte des **souhaits** exprimés par chaque club, tout en garantissant qu'aucun numéro n'est attribué deux fois dans la même poule.

1.2 Problématique

Aujourd'hui, la constitution des poules et l'organisation des rencontres sont encore réalisées manuellement par les responsables de compétition, ce qui représente une tâche longue et exposée aux erreurs. L'absence d'outil dédié contraint les organisateurs à recommencer intégralement le processus à chaque nouvelle saison, aussi bien pour la répartition des équipes en poules que pour la numérotation des équipes en vue de la construction du calendrier.

La question centrale de ce projet est donc la suivante :

Comment concevoir une application web permettant d'automatiser la constitution de poules sportives et la numérotation des équipes en vue de l'élaboration du calendrier des rencontres ?

1.3 Objectifs du projet

Pour répondre à cette problématique, le projet vise à développer une application web entièrement côté client, ne nécessitant aucun serveur, et accessible directement depuis un navigateur. Les principaux objectifs sont les suivants :

1. **Import et lecture de données** : permettre à l'organisateur de déposer ses fichiers de données au format CSV, aussi bien le référentiel des clubs que les listes d'équipes par niveau, et d'obtenir un retour immédiat en cas d'anomalie dans les fichiers fournis.
2. **Algorithmes de répartition** : proposer deux modes de génération complémentaires. Le premier mode minimise les distances géographiques entre les clubs d'une même poule. Le second mode équilibre les poules en fonction du statut sportif des équipes (montantes, stables, descendantes) tout en cherchant à réduire les déplacements.
3. **Visualisation cartographique** : afficher les poules générées sur une carte interactive, en représentant chaque équipe par un marqueur coloré selon sa poule d'appartenance, afin de permettre une lecture géographique immédiate du résultat.
4. **Visualisation tabulaire et statistiques** : présenter les poules sous forme de tableaux détaillés accompagnés de métriques quantitatives (distance moyenne intra-poule, écart-type, poids de niveau), permettant à l'organisateur d'évaluer objectivement la qualité de la répartition obtenue.
5. **Édition manuelle des poules** : offrir un mode d'édition permettant à l'organisateur d'échanger manuellement des équipes entre les poules, avec vérification automatique des contraintes réglementaires et mise à jour instantanée des statistiques.
6. **Numérotation des équipes** : à partir des poules générées et des souhaits exprimés par les clubs, attribuer automatiquement un numéro à chaque équipe en respectant trois types de contraintes : numéro identique pour toutes les équipes d'un même club (*synchronise*), numéros opposés entre les groupes d'équipes d'un même club (*reparti*), ou attribution libre (*sans_preference*).

Chapitre 2

Organisation du stage

2.1 Diagramme de Gantt

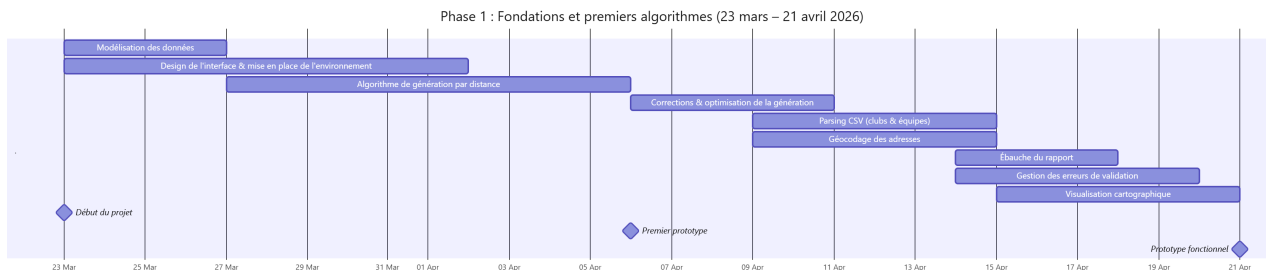


Figure 2.1 – Diagramme de Gantt phase 1

2.2 Organisation et répartition du travail

Le projet a été conduit en binôme selon une organisation régulière. Deux réunions hebdomadaires minimum ont été tenues afin de faire le point sur l'avancement de chacun, d'identifier les éventuels blocages et de redistribuer les tâches en fonction des priorités du moment. Ces tâches étaient réparties selon les fonctionnalités à développer, chacun travaillant de manière autonome entre les réunions.

Le versionnement du code a été assuré via Git, hébergé sur un dépôt GitHub commun. Cet outil a permis de travailler en parallèle sur des branches distinctes, de fusionner les contributions de chacun et de conserver un historique complet de l'évolution du projet.

Lien du répertoire github : [TER 2026 Constitution de poules sportives](#).

Chapitre 3

Modélisation des données et fichiers d'entrée

3.1 Modélisation des données

L'application repose sur trois entités principales : le **club**, qui constitue le référentiel géographique, l'**équipe**, qui est l'unité de répartition manipulée par les algorithmes, et la **poule**, qui est le résultat produit. Le diagramme de classes ci-dessous illustre leurs relations.

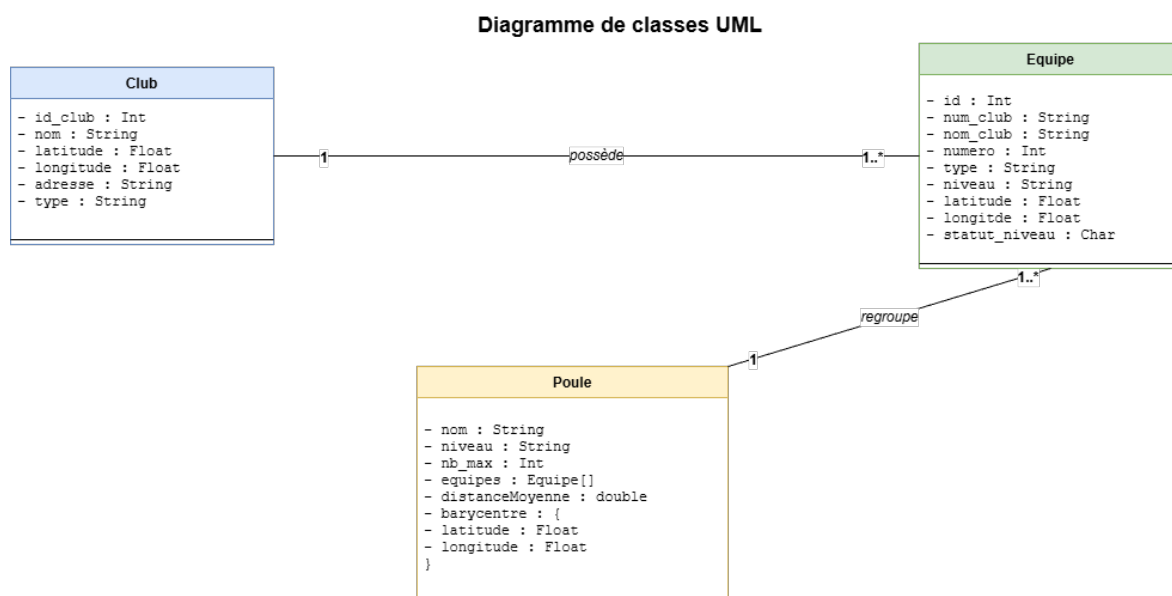


Figure 3.1 – Diagramme de classes du modèle de données.

Club

Un club est issu du fichier de référentiel fourni par la FFBB et géocodé au préalable. Il sert uniquement de source de coordonnées géographiques : chaque équipe hérite de la

position de son club.

```
1 {
2   id_club:    "PDL0049002",
3   nom:        "AUTHION ENTENTE BASKET",
4   latitude:   47.4784,           // WGS84
5   longitude:  -0.5632,           // WGS84
6   adresse:    "20 RUE ARMAND QUENARD 49650 ALLONNES",
7   type:       "Club"
8 }
```

Listing 3.1 – Structure d'un objet club

Cas particulier — Les Coopérations Territoriales de Clubs (CTC)

Une **Coopération Territoriale de Club (CTC)** est un accord entre plusieurs clubs d'un même territoire qui s'associent pour former une équipe commune, généralement dans le but de maintenir une participation à un niveau de compétition qu'aucun des clubs ne pourrait assurer seul.

Dans le fichier des clubs fourni par la FFBB, une CTC apparaît comme une entrée ordinaire — elle possède exactement les mêmes attributs qu'un club classique.

```
1 {
2   id_club:    "PDL0049142",
3   nom:        "CTC DU JEU",
4   latitude:   47.269057,
5   longitude:  -0.824651,
6   adresse:    "15 rue de l'Avenir, 49120 Neuvy-en-Mauges",
7   type:       "Coopération Territoriale Club"
8 }
```

Listing 3.2 – Structure d'un objet club de type CTC

Équipe

Une équipe est construite à partir du fichier CSV de niveau. Elle embarque les coordonnées de son club, qui sont utilisées par les algorithmes de génération. Les attributs `ctc_nom` et `ctc_num` ne sont présents que pour les équipes de type CTC. L'attribut `distance_totale` est calculé après la génération des poules.

```
1 {
2   id:          "AUTHION ENTENTE BASKET-1",
3   num_club:    "PDL0049002",
4   nom_club:    "AUTHION ENTENTE BASKET",
5   numero:      1,
6   type:        "Club",           // "Club" ou "CTC"
```

```
7   niveau:           "D1",
8   statut_niveau:    "=",                // "-", "=" ou "+"
9   latitude:         47.4784,
10  longitude:        -0.5632,
11  distance_totale: 0                    // calculé apres generation
12  // Attributs supplementaires pour une equipe de type CTC :
13  // ctc_nom: "NOM DE LA CTC",
14  // ctc_num: "CTC-001"
15 }
```

Listing 3.3 – Structure d'un objet équipe (type Club)

Poule

Une poule est initialisée avant la génération, puis enrichie au fur et à mesure que les équipes lui sont affectées. Les statistiques (`distance_moyenne`, `ecart_type`) sont calculées et mises à jour à chaque modification de la composition de la poule.

```
1 {
2   nom:              "A",
3   niveau:           1,
4   nb_max:           5,                // capacite maximale
5   equipes:          [],              // tableau d'objets equipe
6   barycentre:       { latitude: 47.4, longitude: -0.5 },
7   distance_moyenne: 0,                // km, moyenne inter-equipes
8   ecart_type:       0                 // ecart-type des distances au
9   barycentre
10 }
```

Listing 3.4 – Structure d'un objet poule

3.2 Données en entrée de l'application

Tous les fichiers d'entrée de l'application sont au format **CSV** avec une **virgule** comme séparateur. La première ligne de chaque fichier contient les en-têtes des colonnes.

3.2.1 Fichier des clubs

Le fichier des clubs constitue le référentiel géographique de l'application. Fourni par la Fédération Française de Basketball (FFBB), il recense l'ensemble des clubs avec leurs informations administratives et leurs coordonnées géographiques. Ce fichier doit être préalablement géocodé afin d'enrichir chaque entrée d'une latitude et d'une longitude, indispensables au calcul des distances et à la génération des poules.

Les colonnes exploitées par l'application sont les suivantes :

Colonne	Description	Obligatoire
cd_org	Identifiant unique du club (ex. PDL0049000)	Oui
lb_org	Nom du club	Oui
adresse	Rue et numéro	Non
adresse_complement	Complément d'adresse	Non
lb_cmne	Commune	Non
code_postal	Code postal	Non
lb_type_association	Type d'association	Non
longitude	Longitude WGS84 (issue du géocodage)	Oui
latitude	Latitude WGS84 (issue du géocodage)	Oui

Table 3.1 – Colonnes du fichier des clubs exploitées par l'application.

Les colonnes supplémentaires issues du géocodage (score, libellé, type de résultat, etc.) sont ignorées par l'application.

3.2.2 Fichier des équipes

Pour chaque niveau du championnat, l'organisateur dépose un fichier contenant la liste des équipes participantes. Ce fichier est la principale source de données de l'application : c'est à partir de lui que les équipes sont extraites, validées et transmises à l'algorithme de génération des poules.

Les colonnes exploitées par l'application sont les suivantes :

Colonne	Description	Obligatoire
CLUB_NOM	Nom du club porteur	Oui
CLUB_NUMERO	Numéro unique du club porteur	Oui
TYPE_EQUIPE	Type : <i>Club</i> ou <i>CTC</i>	Oui
CTC_NOM	Nom de la CTC	Si CTC
CTC_NUMERO	Numéro unique de la CTC	Si CTC
EQUIPE_NUMERO	Numéro de l'équipe au sein du club	Oui
NIVEAU	Niveau de compétition (ex : D1, D2)	Mode niveau
STATUT_NIVEAU	Statut : montant, descendant ou stable	Mode niveau

Table 3.2 – Colonnes du fichier des équipes exploitées par l'application.

Les colonnes restantes du fichier (informations administratives, montants d'engagement, etc.) sont ignorées par l'application.

3.2.3 Fichier des poules

Ce fichier est produit par l'application à l'issue de la génération des poules et sert de fichier d'entrée pour l'étape de numérotation des équipes.

Les colonnes exploitées par l'application sont les suivantes :

Colonne	Description	Obligatoire
Catégorie	Catégorie du championnat (ex : senior)	Oui
Niveau	Niveau de compétition (entier)	Oui
Poule	Lettre identifiant la poule (ex : A, B, C)	Oui
Équipe	Nom et numéro de l'équipe (ex : ANDARD-1)	Oui
Club	Numéro du club	Oui

Table 3.3 – Colonnes du fichier des poules générées.

Après lecture du fichier, les données sont organisées dans la structure suivante, indexée par niveau :

```
1 {
2   1: {
3     categorie: "senior",
4     poules: {
5       "A": [
6         { nomEquipe: "VALLEE BASKET CLUB-1",
7           numClub: "PDL0049117", numeroAttribue: null },
8         { nomEquipe: "ANDARD BRAIN-1",
9           numClub: "PDL0049004", numeroAttribue: null },
10        { nomEquipe: "Exempt",
11          numClub: "-", numeroAttribue: null }
12      ],
13      "B": [
14        { nomEquipe: "SAINT CHRISTOPHE DU BOIS-1",
15          numClub: "PDL0049097", numeroAttribue: null },
16        { nomEquipe: "PUY SAINT BONNET BASKET-1",
17          numClub: "PDL0049093", numeroAttribue: null },
18        { nomEquipe: "MURS ERIGNE BASKET-1",
19          numClub: "PDL0049082", numeroAttribue: null }
20      ]
21    }
22  },
23  2: { ... }
24 }
```

Listing 3.5 – Structure de données des poules après lecture du fichier

L'attribut `numeroAttribue` est initialisé à `null` pour toutes les équipes et sera renseigné lors de l'étape de numérotation.

Les lignes *Exempt* sont présentes dans le fichier pour les poules dont le nombre d'équipes est inférieur à la taille maximale du niveau. Leur présence permet à l'algorithme de numérotation de déduire la taille commune T de toutes les poules d'un niveau donné.

3.2.4 Fichier des souhaits des clubs

Ce fichier est fourni par l'organisateur et constitue la base de l'algorithme de numérotation. Il associe à chaque club, identifié par son numéro, le souhait exprimé concernant la répartition de ses équipes entre les différents numéros. Un club absent de ce fichier est traité comme `sans_preference`.

```
1 Groupe Sportive, Souhait
2 PDL0049031, sans_preference
3 PDL0049115, sans_preference
4 PDL0049126, synchronise
5 PDL0049084, reparti
6 PDL0049170, sans_preference
```

Listing 3.6 – Exemple de fichier des souhaits des clubs

Valeur	Signification
sans_preference	Aucune contrainte sur le numéro attribué
synchronise	Toutes les équipes du club reçoivent le même numéro
reparti	Les équipes du club reçoivent des numéros opposés

Table 3.4 – Valeurs possibles du champ `Souhait` dans le fichier des souhaits.

Chapitre 4

Conception des algorithmes de répartition des équipes

4.1 Étapes préliminaires communes

4.1.1 Vérifications préalables

Avant de lancer la génération des poules, le système effectue trois vérifications sur les paramètres saisis par l'organisateur. Ces vérifications permettent de détecter immédiatement toute configuration impossible ou incohérente, et d'informer l'utilisateur avec un message d'erreur précis.

Vérification 1 : Capacité totale suffisante

La première vérification s'assure que le nombre total de places disponibles dans toutes les poules est suffisant pour accueillir toutes les équipes.

Condition :

$$nb_poules \times nb_max \geq nb_equipes$$

Si cette condition n'est pas respectée, certaines équipes ne pourront pas être placées. L'organisateur doit alors soit augmenter le nombre de poules, soit augmenter la taille maximale par poule.

Exemple : 50 équipes, 8 poules de 5 $\rightarrow$ capacité totale = 40 < 50 $\rightarrow$ **refusé**

Vérification 2 : Équilibre des tailles de poules

La deuxième vérification garantit que les poules générées seront équilibrées. On accepte qu'une poule ait au plus **un exempt** (une place vide), mais on exige qu'au moins une poule soit **complètement pleine**.

Cela revient à vérifier que le nombre d'équipes est strictement supérieur au nombre total de places si chaque poule avait une équipe de moins que le maximum.

Condition :

$$nb_equipes > nb_poules \times (nb_max - 1)$$

Si cette condition n'est pas respectée, toutes les poules auraient un exempt, ce qui signifie que l'organisateur aurait pu choisir une taille maximale inférieure dès le départ.

Exemple : 15 équipes, 3 poules de 8 $\rightarrow 3 \times 8 = 24 \geq 15 \rightarrow$ toutes les poules auraient 3 places vides $\rightarrow$ **refusé**, l'organisateur devrait choisir 3 poules de 5 directement.

Vérification 3 : Contrainte de club

La troisième vérification s'assure qu'il sera possible de répartir les équipes en respectant la règle fondamentale : **deux équipes d'un même club ne peuvent pas être dans la même poule**.

Pour que cette règle soit toujours respectée, il faut que le club ayant le plus d'équipes n'en ait pas plus que le nombre de poules disponibles. En effet, si un club a plus d'équipes que de poules, il est mathématiquement impossible de les séparer.

Condition :

$$\max_{c \in \text{clubs}} (nb_equipes(c)) \leq nb_poules$$

Exemple : un club avec 5 équipes, 4 poules $\rightarrow$ impossible de placer ces 5 équipes dans 4 poules sans conflit $\rightarrow$ **refusé**

Synthèse

Le tableau 4.1 résume les trois vérifications dans l'ordre où elles sont effectuées.

#	Condition vérifiée	Message si non respectée
1	$nb_poules \times nb_max \geq nb_equipes$	Capacité insuffisante
2	$nb_equipes > nb_poules \times (nb_max - 1)$	Trop peu d'équipes
3	$\max(\text{équipes par club}) \leq nb_poules$	Contrainte club impossible

Table 4.1 – Synthèse des vérifications préalables à la génération des poules

Ces trois vérifications sont **bloquantes** : si l'une d'elles échoue, la génération est immédiatement interrompue et un message explicite est affiché à l'organisateur.

4.1.2 Définition des capacités des poules

Avant tout placement, on détermine la capacité maximale de chaque poule. Si le nombre d'équipes n'est pas divisible exactement par le nombre de poules, certaines poules auront une équipe de moins que les autres (un exempt).

Soit n le nombre d'équipes et p le nombre de poules :

$$base = \left\lfloor \frac{n}{p} \right\rfloor \quad reste = n \bmod p$$

Les *reste* premières poules reçoivent une capacité de $base + 1$, les autres reçoivent $base$. Pour éviter que ce soient toujours les mêmes poules qui bénéficient d’une place supplémentaire, les capacités sont ensuite **mélangées aléatoirement** grâce à l’algorithme de Fisher-Yates :

Pour i de $(p - 1)$ à 1 : choisir j aléatoirement dans $[0, i]$, puis échanger $capacites[i]$ et $capacites[j]$.

4.1.3 Initialisation des poules

Une fois les capacités définies, les poules sont initialisées comme des structures vides prêtes à accueillir les équipes. Chaque poule est représentée par un objet contenant les attributs suivants :

Attribut	Description	Valeur initiale
nom	Lettre identifiant la poule (A, B, C, ...)	A, B, C, ...
niveau	Niveau de compétition de la poule	Niveau courant
nb_max	Capacité maximale de la poule	Définie en 4.1.2
equipes	Liste des équipes affectées à la poule	[] (vide)
distance_moyenne	Distance moyenne entre les équipes de la poule	0
barycentre	Centre géographique de la poule	null

Table 4.2 – Attributs d’une poule à l’initialisation

Quelques remarques sur ces attributs :

- **Nom** : les poules sont nommées par ordre alphabétique — la première poule s’appelle A, la deuxième B, et ainsi de suite. Ce nommage est purement indicatif et facilite l’affichage dans l’interface.
- **nb_max** : chaque poule reçoit sa propre capacité issue du mélange aléatoire décrit en section [4.1.2](#) — certaines poules auront une capacité de $base + 1$ et d’autres une capacité de $base$.
- **equipes** : le tableau est vide à l’initialisation. La taille de ce tableau est utilisée tout au long de l’algorithme pour vérifier si une poule est pleine ($|equipes| \geq nb_max$), sans avoir recours à un compteur séparé.
- **distance_moyenne et barycentre** : ces deux attributs sont calculés et mis à jour dynamiquement après chaque ajout d’une équipe dans la poule. Ils valent respectivement 0 et null tant que la poule est vide.

4.2 Algorithme de répartition par distance

L'algorithme de génération par distance est le cœur du système. Son objectif est de répartir les équipes en poules de façon à **minimiser les distances parcourues** par les équipes tout en respectant la contrainte de club. Il se déroule en quatre phases successives.

4.2.1 Sélection des graines — K-Means++

La première étape du placement consiste à choisir p équipes "**graines**", une par poule, qui serviront de points d'ancrage géographiques. Ces graines déterminent la répartition géographique initiale des poules — un mauvais choix de graines conduit inévitablement à des poules déséquilibrées.

Pourquoi ne pas utiliser le barycentre global ?

Une approche naïve consisterait à prendre les p équipes les plus éloignées du barycentre global. Cette approche est insuffisante car ces équipes peuvent être proches entre elles. Par exemple, si la majorité des équipes se trouve au nord, le barycentre est décalé vers le nord, et les équipes les plus éloignées de ce barycentre sont toutes au sud — mais elles peuvent être très proches les unes des autres, ce qui donnerait des poules mal réparties.

Choix de K-Means++ plutôt que K-Means classique

L'algorithme K-Means classique, introduit par Lloyd [1], initialise les p graines de façon **aléatoire uniforme** — chaque équipe a la même probabilité d'être choisie comme graine, indépendamment de sa position géographique. Cette approche présente un inconvénient majeur : si les équipes sont **inégalement réparties** sur le territoire, les graines peuvent se retrouver concentrées dans les zones les plus denses, laissant certaines zones géographiques sans graine de référence.

Exemple : avec 20 équipes dont 15 situées dans la région d'Angers et 5 dans la région de Cholet, K-Means classique a une probabilité de $\frac{15}{20} = 75\%$ de choisir chaque graine dans la région d'Angers. Avec $p = 4$ graines, la probabilité que toutes soient dans la région d'Angers est de $\left(\frac{15}{20}\right)^4 \approx 31.6\%$ — soit près d'une chance sur trois d'obtenir une initialisation très défavorable où la région de Cholet n'est représentée par aucune graine.

L'algorithme **K-Means++**, proposé par Arthur et Vassilvitskii en 2007 [2], résout ce problème en sélectionnant les graines de façon **déterministe et progressive**. La première graine est choisie comme l'équipe la plus éloignée du barycentre global, puis

chaque graine suivante est choisie comme l'équipe maximisant sa distance à la graine la plus proche déjà sélectionnée :

$$G_i = \arg \max_{E \notin \{G_0, \dots, G_{i-1}\}} \min_{j < i} \text{dist}(E, G_j)$$

Cette stratégie garantit que les graines sont **géographiquement bien réparties** sur le territoire dès l'initialisation, ce qui réduit le nombre d'itérations nécessaires dans les phases suivantes et améliore la qualité finale des poules.

Le tableau 4.3 résume les différences entre les deux approches.

Critère	K-Means classique	K-Means++
Initialisation des graines	Aléatoire uniforme	Déterministe (max-min dist)
Risque de concentration	Élevé	Quasi nul
Qualité initiale	Variable selon le hasard	Toujours bien répartie
Convergence	Lente si mauvaise init.	Rapide
Complexité d'init.	$O(p)$	$O(p \times n)$

Table 4.3 – Comparaison entre K-Means classique et K-Means++

La légère surcharge computationnelle de K-Means++ ($O(p \times n)$ au lieu de $O(p)$) est largement compensée par la qualité de l'initialisation obtenue, notamment dans notre contexte où les équipes peuvent être très inégalement réparties sur le territoire.

La solution : K-Means++

L'algorithme K-Means++ résout ce problème en sélectionnant les graines de façon à **maximiser mutuellement leur éloignement**.

- **Graine 0** : on calcule le barycentre global de toutes les équipes (unique calcul de barycentre dans cette phase), et on choisit l'équipe la plus éloignée de ce barycentre comme point de départ.
- **Graines suivantes** : pour chaque nouvelle graine G_i , on évalue chaque équipe candidate E avec la formule :

$$d[E] = \min(\text{dist}(E, G_0), \text{dist}(E, G_1), \dots, \text{dist}(E, G_{i-1}))$$

On choisit l'équipe dont $d[E]$ est **maximal** — c'est l'équipe la plus isolée de toutes les graines déjà placées.

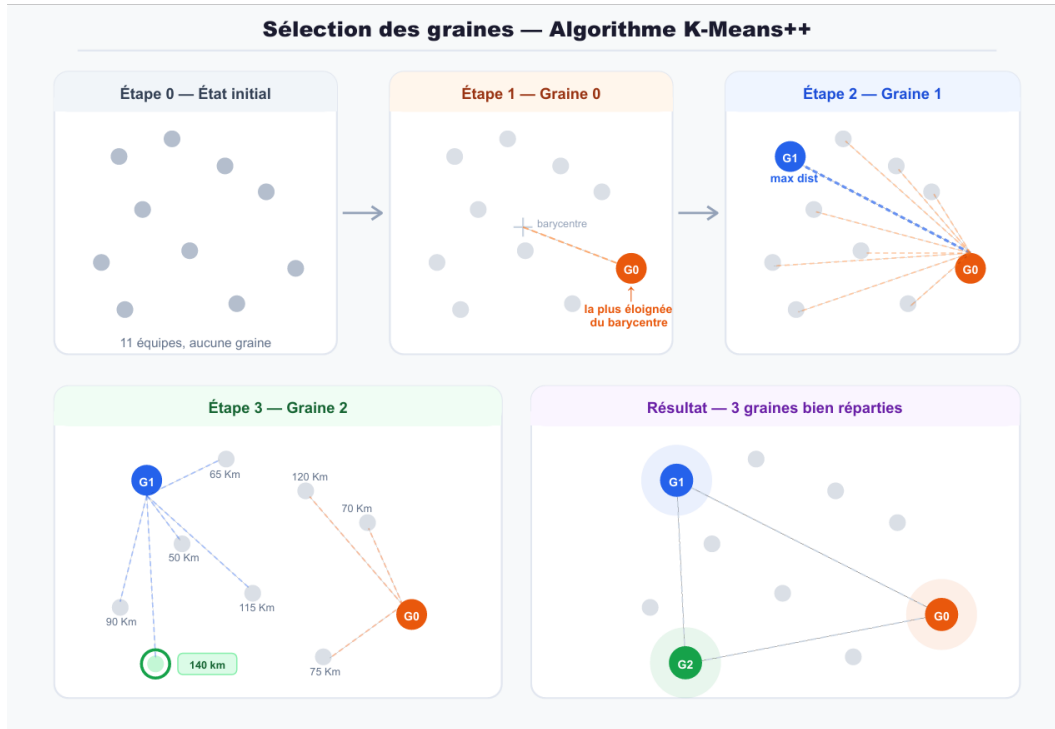


Figure 4.1 — Sélection des graines par l’algorithme K-Means++

Une fois les p graines sélectionnées, chacune est placée dans une poule distincte. Cela garantit que les poules sont initialisées dans des zones géographiques bien séparées.

4.2.2 Affectation des équipes restantes

Les équipes non sélectionnées comme graines sont ensuite affectées une par une. Elles sont d’abord **triées par ordre décroissant de leur distance au barycentre global** — les équipes les plus isolées géographiquement sont traitées en priorité, quand les poules ont encore de la flexibilité pour les accueillir.

Pour chaque équipe E à placer, on parcourt toutes les poules et on applique le processus de décision suivant :

1. **La poule est-elle pleine ?** Si $|P| \geq nb_max \rightarrow$ on passe à la suivante.
2. **Y a-t-il un conflit de club ?** Si la poule contient déjà une équipe du même club que $E \rightarrow$ on passe à la suivante.
3. **Calcul de proximité :** on évalue la compatibilité géographique de E avec la poule selon le critère retenu (voir ci-dessous).
4. On retient la poule dont le score est **minimal**.

Choix du critère de sélection de la meilleure poule

Lors de l’affectation d’une équipe E à une poule, deux critères de sélection sont envisageables. Nous les comparons ici pour justifier notre choix.

Critère 1 — Barycentre le plus proche On choisit la poule dont le barycentre géographique est le plus proche de l'équipe à placer :

$$P^* = \arg \min_{P \in \mathcal{P}} \text{dist}(E, \text{bary}(P))$$

Avantage : très rapide — un seul calcul de distance par poule candidate, soit une complexité de $O(p)$ pour placer une équipe.

Inconvénient : le barycentre est une approximation du centre géographique de la poule. Il ne reflète pas nécessairement l'impact réel de l'ajout de E sur la cohésion de la poule.

Critère 2 — Distance moyenne minimale après ajout On simule l'ajout de E dans chaque poule candidate et on retient celle dont la distance moyenne après ajout est minimale :

$$P^* = \arg \min_{P \in \mathcal{P}} \bar{d}(P \cup \{E\})$$

Avantage : mesure directement l'impact réel de l'ajout sur la cohésion géographique de la poule.

Inconvénient : ce critère présente un biais important. Considérons l'exemple suivant :

Poule	Dist. moy. avant	Dist. moy. après	Augmentation
P_A	30 km	32 km	+2 km
P_B	15 km	28 km	+13 km

Table 4.4 – Exemple illustrant le biais du critère 2

Le critère 2 choisit P_B car $28 < 32$, alors que l'ajout dans P_A ne dégrade sa cohésion que de 2 km contre 13 km pour P_B . Le critère 2 **favorise systématiquement les poules déjà denses** au détriment des poules géographiquement plus dispersées, accentuant ainsi les déséquilibres entre poules.

Critère 3 — Augmentation minimale de la distance moyenne On choisit la poule pour laquelle l'ajout de E dégrade le moins la cohésion géographique, c'est-à-dire celle dont l'augmentation de la distance moyenne est minimale :

$$P^* = \arg \min_{P \in \mathcal{P}} (\bar{d}(P \cup \{E\}) - \bar{d}(P))$$

Avantage : évite le biais du critère 2 — une poule déjà dense ne sera pas systématiquement favorisée. On mesure l'impact réel de l'ajout sur chaque poule indépendamment de son état actuel.

Inconvénient : comme le critère 2, nécessite de simuler l’ajout dans chaque poule candidate, soit une complexité de $O(p \times n)$.

Discussion et choix retenu Nos tests expérimentaux ont montré que les trois critères produisent des résultats similaires en pratique. En effet, la phase de rééquilibrage (section 4.2.4) corrige les déséquilibres introduits lors de l’affectation quel que soit le critère utilisé — le nombre d’itérations de rééquilibrage restant sensiblement le même dans les trois cas. Nous avons donc retenu le **critère 1 (barycentre le plus proche)** pour sa simplicité et son efficacité computationnelle. Notons que le barycentre de chaque poule est **recalculé et mis à jour après chaque ajout**, ce qui atténue la limite du critère 1 — le barycentre reflète à tout moment la composition réelle de la poule et non sa configuration initiale.

4.2.3 Stratégie de sauvetage

Dans certaines configurations, une équipe ne peut être placée dans aucune poule : toutes les poules sont soit pleines, soit contiennent déjà une équipe du même club. On applique alors une **stratégie de sauvetage par échange**.

Le principe est de trouver une poule P qui n’a pas de conflit avec le club de l’équipe bloquée E , d’en extraire une équipe $E_{\text{échange}}$ et de la déplacer vers une poule non pleine P_{cible} , libérant ainsi une place dans P pour E .

Pour choisir le **meilleur échange** parmi tous les échanges valides, on calcule un score combiné :

$$\text{score} = \text{dist}(E, \text{bary}_{P_{\text{apres}}}) + \text{dist}(E_{\text{échange}}, \text{bary}_{P_{\text{cible}}})$$

- Le premier terme mesure la qualité de l’accueil de E dans P après le retrait de $E_{\text{échange}}$.
- Le second terme mesure la qualité de l’accueil de $E_{\text{échange}}$ dans P_{cible} .

On retient l’échange dont le score est **minimal** — les deux équipes sont ainsi placées dans les meilleures conditions géographiques possibles.

4.2.4 Rééquilibrage des distances moyennes

Même avec un algorithme glouton bien conçu, des déséquilibres peuvent subsister entre les poules après la phase d’affectation. Une poule peut se retrouver avec une distance moyenne bien supérieure aux autres, créant ainsi une inégalité sportive entre les équipes. On effectue donc une phase de **rééquilibrage par échanges améliorants**.

Principe

L'idée est simple : on cherche, parmi tous les échanges possibles entre deux équipes appartenant à deux poules différentes, celui qui **réduit le plus la somme des distances moyennes** des deux poules concernées. On applique cet échange, puis on recommence jusqu'à ce qu'aucun échange n'améliore la situation.

Critère d'évaluation

Pour chaque paire de poules (P_A, P_B) et chaque paire d'équipes $(e_A \in P_A, e_B \in P_B)$, on simule l'échange et on calcule le gain :

$$gain = (\bar{d}(P_A) + \bar{d}(P_B)) - (\bar{d}(P'_A) + \bar{d}(P'_B))$$

où :

- $\bar{d}(P)$ désigne la distance moyenne actuelle de la poule P
- P'_A désigne P_A après remplacement de e_A par e_B
- P'_B désigne P_B après remplacement de e_B par e_A

Un gain positif signifie que l'échange réduit la somme des distances moyennes des deux poules — c'est un échange améliorant. On retient l'échange dont le gain est **maximal** parmi tous les échanges valides de l'itération.

Contrainte de club

Avant d'évaluer un échange, on vérifie que la contrainte de club est respectée dans les deux sens :

- e_B ne doit pas appartenir au même club qu'une équipe déjà présente dans P_A (hors e_A qui part)
- e_A ne doit pas appartenir au même club qu'une équipe déjà présente dans P_B (hors e_B qui part)

Si l'une de ces conditions n'est pas respectée, l'échange est ignoré et on passe au suivant.

Déroulement de l'algorithme

1. Pour chaque paire de poules (P_A, P_B) avec $a < b$:
 - (a) Pour chaque paire d'équipes (e_A, e_B) :
 - i. Vérifier la contrainte de club dans les deux sens
 - ii. Calculer $\bar{d}(P'_A)$ et $\bar{d}(P'_B)$ par simulation
 - iii. Calculer le gain et mémoriser si c'est le meilleur jusqu'ici

2. Si aucun échange améliorant n'est trouvé → **convergence**, arrêt
3. Sinon, appliquer le meilleur échange et passer à l'itération suivante

Le nombre d'itérations est plafonné à **100** pour garantir la terminaison de l'algorithme dans tous les cas.

Complexité

À chaque itération, on évalue toutes les paires de poules et toutes les paires d'équipes. Grâce à la condition $a < b$ dans le code, chaque paire de poules n'est traitée qu'une seule fois, ce qui donne $\binom{p}{2}$ paires au lieu de p^2 .

Le nombre de paires de poules est :

$$\binom{p}{2} = \frac{p \times (p - 1)}{2}$$

Exemple avec $p = 4$ poules $\{A, B, C, D\}$, les paires traitées sont :

$$(A, B), (A, C), (A, D), (B, C), (B, D), (C, D) \Rightarrow \frac{4 \times 3}{2} = 6 \text{ paires}$$

Les paires symétriques (B, A) , (C, A) , etc. ne sont jamais évaluées car l'échange (P_A, P_B) est identique à l'échange (P_B, P_A) .

Pour chaque paire de poules, on teste toutes les combinaisons d'équipes ($e_A \in P_A$, $e_B \in P_B$), soit n^2 combinaisons si n désigne le nombre moyen d'équipes par poule.

La complexité d'une itération est donc :

$$\binom{p}{2} \times n^2 = \frac{p(p-1)}{2} \times n^2 \sim \frac{p^2}{2} \times n^2 = O(p^2 \times n^2)$$

car en notation O , les constantes multiplicatives sont ignorées.

Dans le contexte d'un championnat de basketball régional, p et n restent faibles (typiquement $p \leq 20$ et $n \leq 10$), ce qui rend cette complexité tout à fait acceptable en pratique.

Convergence

L'algorithme converge car à chaque itération la somme des distances moyennes de toutes les poules **diminue strictement** (on n'applique un échange que si le gain est strictement positif). Cette somme étant bornée inférieurement par zéro, l'algorithme se termine nécessairement en un nombre fini d'itérations.

4.3 Algorithme de répartition par niveau

4.3.1 Prérequis

Données

La répartition par niveau repose sur un attribut **statut_niveau** associé à chaque équipe, qui reflète son évolution sportive lors de la saison précédente. Cet attribut prend l'une des trois valeurs suivantes :

- **"-"** : l'équipe est **descendante** c'est à dire qu'elle provient du niveau supérieur et est supposée plus forte que la moyenne du niveau courant.
- **"="** : l'équipe est **stable** c'est à dire qu'elle évolue au même niveau que la saison précédente.
- **"+"** : l'équipe est **montante** c'est à dire qu'elle provient du niveau inférieur et est supposée moins forte que la moyenne du niveau courant.

On a donc la relation de force supposée : équipes **"-"** $\succ$ équipes **"="** $\succ$ équipes **"+"**. L'objectif de l'algorithme est de construire des poules dont le niveau de difficulté est aussi homogène que possible, en répartissant équitablement les équipes de chaque statut.

Opérations préliminaires

Une fois les données lues depuis le fichier CSV correspondant, les capacités de chaque poule sont définies conformément à la section 4.1.2 et les poules sont initialisées conformément à la section 4.1.3.

4.3.2 Calcul des quotas par statut et par poule

L'objectif du calcul des quotas est de déterminer, pour chaque poule, le nombre cible d'équipes de chaque statut, de façon à garantir un équilibre de difficulté entre les poules.

Ce calcul est effectué indépendamment pour chacun des trois statuts. Pour un statut donné comptant n équipes à répartir dans p poules, la distribution cible est :

$$\text{base} = \left\lfloor \frac{n}{p} \right\rfloor, \quad \text{reste} = n \bmod p$$

Le résultat est un vecteur de quotas Q de taille p , où :

$$Q[i] = \begin{cases} \text{base} + 1 & \text{si } i < \text{reste} \\ \text{base} & \text{sinon} \end{cases} \quad \text{pour } i \in \{0, \dots, p-1\}$$

Ainsi, les reste premières poules reçoivent base + 1 équipes de ce statut, et les suivantes en reçoivent base. Cette règle garantit que les quotas diffèrent d'au plus une unité entre deux poules pour un même statut, et que la somme des quotas est exactement égale à n .

4.3.3 Distribution des équipes

Cette étape répartit les équipes dans les poules en respectant simultanément trois contraintes : les quotas par statut, la contrainte d'unicité (au plus une équipe par club dans une même poule), et la capacité maximale des poules.

Le traitement est effectué par statut, dans l'ordre décroissant d'impact sur la difficulté des poules : les équipes de statut "-" en premier, puis celles de statut "+", et enfin celles de statut "=", qui ont le moins d'influence sur l'équilibre. Cet ordre de priorité permet de placer les cas extrêmes en premier, là où les contraintes sont les plus fortes, et de réserver les ajustements aux équipes neutres.

Pour chaque statut, les équipes sont affectées à tour de rôle dans les poules dont le quota restant est non nul. À chaque placement, le quota de la poule destinataire est décrémenté. Si toutes les poules ayant un quota disponible contiennent déjà une équipe du même club, un **placement de secours** est effectué : l'équipe est assignée à la poule non pleine la moins remplie, sans affecter les quotas.

4.3.4 Optimisation géographique

Une fois la répartition par statut établie, une phase d'optimisation cherche à réduire les distances parcourues par les équipes tout en préservant l'équilibre de difficulté obtenu à l'étape précédente.

L'optimisation procède par échanges successifs entre paires de poules : pour chaque paire de poules (A, B) et chaque paire d'équipes (e_A, e_B) candidates à l'échange, le gain en distance moyenne est évalué. L'échange n'est retenu que si les deux conditions suivantes sont satisfaites :

- e_A et e_B ont le **même statut**, garantissant l'invariance du niveau global de chaque poule après l'échange.
- Ni e_A dans la poule B , ni e_B dans la poule A ne créent de doublon de club.

Les échanges améliorants sont appliqués itérativement, jusqu'à ce qu'aucun gain supplémentaire ne soit possible ou que le nombre maximal de passes (20) soit atteint.

Chapitre 5

Numérotation des équipes

5.1 Prérequis — Uniformité des tailles de poules

L'algorithme exige que **toutes les poules aient exactement le même nombre d'équipes** T . Cette condition est vérifiée avant tout traitement : si des poules de tailles différentes sont détectées, la numérotation est immédiatement interrompue et un message d'erreur est affiché à l'organisateur.

5.2 Notion de numéro opposé

Pour une poule de taille T , et pour tout numéro $n \in [1 \dots \frac{T}{2}]$, le **numéro opposé** de n est défini par :

$$n_{\text{opposé}} = n + \frac{T}{2}$$

Exemple avec $T = 6$: le numéro opposé de 2 est $2 + 3 = 5$. De façon générale, les numéros $[1 \dots T/2]$ et les numéros $[T/2 + 1 \dots T]$ forment deux groupes de numéros opposés deux à deux :

$$n \in [1 \dots \frac{T}{2}] \quad \longleftrightarrow \quad n + \frac{T}{2} \in [\frac{T}{2} + 1 \dots T]$$

Un club souhaitant une répartition équilibrée verra ses équipes divisées en deux groupes : toutes les équipes du premier groupe reçoivent le même numéro n , et toutes les équipes du second groupe reçoivent le numéro opposé $n + T/2$.

5.3 Algorithme de numérotation

L'algorithme traite les clubs dans un ordre précis, du plus contraint au moins contraint. Pour chaque poule, on maintient un ensemble de **slots occupés** qui mémorise

les numéros déjà attribués, garantissant ainsi l'unicité de chaque numéro dans une poule.

Étape 1 — Clubs **synchronise**

Les clubs ayant choisi **synchronise** sont traités en premier car ils imposent la contrainte la plus forte : toutes leurs équipes doivent recevoir **le même numéro** dans toutes les poules où elles apparaissent.

Pour chaque club C de type **synchronise** :

1. On parcourt les numéros $n \in [1, T]$.
2. Pour chaque n , on vérifie qu'il est **libre simultanément** dans toutes les poules où C a une équipe.
3. Dès qu'un tel numéro est trouvé, on l'attribue à toutes les équipes de C et on marque le slot occupé dans chaque poule concernée.
4. Si aucun numéro n'est disponible partout simultanément, **conflit détecté**.

Cas particulier : un club **synchronise** avec une seule équipe est traité comme **sans_preference** à l'étape 3.

Étape 2 — Clubs **reparti**

Les clubs ayant choisi **reparti** sont traités ensuite. La contrainte est que leurs équipes doivent être réparties en deux groupes de numéros opposés : toutes les équipes du Groupe 1 reçoivent le même numéro n , et toutes les équipes du Groupe 2 reçoivent le numéro opposé $n + T/2$.

Si le nombre d'équipes nb est pair, les deux groupes ont la même taille $nb/2$. Si nb est impair, les deux groupes ont des tailles $\lfloor nb/2 \rfloor$ et $\lceil nb/2 \rceil$.

La difficulté est que la **répartition des équipes entre les deux groupes** influe sur la disponibilité des slots. Une répartition peut être impossible alors qu'une autre fonctionne. L'algorithme teste donc toutes les combinaisons possibles de façon optimisée :

1. On fixe la taille du Groupe 1 à $t = \lfloor nb/2 \rfloor$ et on génère toutes les façons de choisir t équipes parmi les nb équipes du club, soit $\binom{nb}{t}$ combinaisons.
2. Pour chaque combinaison, le Groupe 1 est constitué des t équipes choisies et le Groupe 2 est le complément.
3. Pour chaque paire de numéros opposés $(n, n + T/2)$ avec $n \in [1 \dots T/2]$, on teste les **deux configurations** :
 - Configuration 1 : numéro n pour le Groupe 1, numéro $n + T/2$ pour le Groupe 2.

- Configuration 2 : numéro $n + T/2$ pour le Groupe 1, numéro n pour le Groupe 2.
- 4. Une configuration est valide si le numéro attribué au Groupe 1 est **libre dans toutes les poules** des équipes de ce groupe, et que le numéro attribué au Groupe 2 est libre dans toutes les poules des équipes de ce groupe simultanément.
- 5. Dès qu’une configuration valide est trouvée, on attribue les numéros et on passe au club suivant.
- 6. Si aucune configuration ne fonctionne pour aucune combinaison, **conflit détecté**.

La génération de toutes les combinaisons est réalisée par une fonction récursive `combinaisons(tableau, k)` qui retourne toutes les façons de choisir k éléments parmi un ensemble de taille n . Elle implémente directement la relation de Pascal :

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$$

La complexité de cette étape est $O\left(\binom{nb}{\lfloor nb/2 \rfloor} \times T/2\right)$ par club, ce qui reste acceptable pour les tailles rencontrées en pratique (nombre d’équipes par club généralement inférieur à 10).

Cas particulier : un club `reparti` avec une seule équipe ne peut pas être réparti — il est traité comme `sans_preference` à l’étape suivante.

Étape 3 — Clubs `sans_preference` et `exempts`

Les équipes restantes (clubs sans préférence, équipes isolées de clubs `reparti`, et lignes *Exempt*) reçoivent les numéros encore disponibles dans leur poule respective, sans contrainte particulière. On parcourt les slots de 1 à T et on attribue le premier slot libre.

5.4 Gestion des conflits

Deux types de conflits peuvent survenir :

1. **Conflit synchronise** : aucun numéro n’est libre simultanément dans toutes les poules des équipes d’un club `synchronise`.
2. **Conflit reparti** : aucune combinaison de répartition ne permet de trouver une paire de numéros opposés disponibles pour toutes les équipes du club.

Dans les deux cas, l’application **n’effectue aucune modification partielle** — elle affiche un message d’erreur précis identifiant le club en conflit et invite l’organisateur à modifier manuellement les souhaits des clubs concernés avant de relancer la numérotation.

Chapitre 6

Choix des technologies utilisées

6.1 Choix du langage : JavaScript natif

Il est globalement attendu de l'application web réalisée qu'elle puisse lire et parser les fichiers en entrée, traiter les données obtenues et présenter les résultats obtenus. Ses opérations peuvent être aisément réalisées côté client, directement dans le navigateur en javascript. Aucun serveur backend n'est nécessaire pour traiter les fichiers CSV, calculer la répartition des poules ou afficher les résultats. JavaScript, étant le langage nativement exécuté par les navigateurs, il convient parfaitement pour cette application.

6.2 Choix de la cartographie : Leaflet.js

Le choix du service de cartographie utilisé s'est articuler autour de quatre (4) lignes directrices qui sont :

- Le type de licence d'usage ;
- Le coût en terme de performances ;
- L'accessibilité ;
- La documentation.

Le tableau ci dessous permet de comparer les solutions les plus populaires en

Table 6.1 – Comparaison des bibliothèques de cartographie web

Critère	Leaflet [3]	Google Maps [4]	OpenLayers [5]	Mapbox [6]
Licence	Open source	Payant (quota)	Open source	Freemium
Poids	~40 Ko	Lourd	~500 Ko	Moyen
Clé API	Non	Oui	Non	Oui
Communauté	Très grande	Très grande	Moyenne	Grande

Ainsi le choix se porte naturellement sur leaflet qui est une bibliothèque javascript open source légère qui correspond parfaitement à l'usage qui en sera fait dans l'application.

6.3 Mise en service via un serveur HTTP local

Bien que l'application soit entièrement côté client et ne nécessite aucun backend pour fonctionner, son exécution requiert qu'elle soit servie par un serveur HTTP, même rudimentaire. Ouvrir directement le fichier `index.html` depuis le système de fichiers (protocole `file://`) est techniquement possible mais déclenche plusieurs restrictions de sécurité imposées par les navigateurs modernes.

Le protocole `file://` traite chaque fichier local comme une origine distincte, ce qui empêche notamment :

- le chargement des modules JavaScript ES déclarés par `<script type="module">`, utilisés pour la séparation du code en plusieurs fichiers (`config.js`, `matrice-distances.js`, `niveau.js`);
- l'envoi de requêtes `fetch` vers l'API de calcul d'itinéraires de l'IGN, indispensable pour la matrice de distances routières;
- le chargement fiable des ressources externes (Leaflet, SheetJS, polices Google).

Servir l'application via un serveur HTTP, même local, attribue à la page une origine valide et stable de la forme `http://localhost:port`, ce qui lève l'ensemble de ces restrictions.

6.3.1 Mise en place

Le serveur retenu pour ce projet est celui fourni nativement par PHP, lancé via la commande :

```
php -S localhost:8000
```

Disponible depuis PHP 5.4, ce serveur intégré est conçu pour le développement et les environnements légers. Il présente plusieurs avantages adaptés au contexte de l'application :

- **Aucune installation supplémentaire** dès lors que PHP est présent sur la machine, configuration courante sur la majorité des systèmes de développement;
- **Démarrage instantané** via une simple commande dans le répertoire racine du projet, sans fichier de configuration;
- **Service de fichiers statiques par défaut**, ce qui suffit à l'application puisqu'aucun traitement côté serveur n'est requis;

- **Légèreté** : le processus consomme très peu de ressources et peut être arrêté immédiatement par **Ctrl+C**.

Une fois le serveur lancé, l'application est accessible à l'adresse **http://localhost:8000** depuis n'importe quel navigateur. Le port peut être modifié librement (par exemple `php -S localhost:3000`) en cas de conflit avec un autre service.

Bien que PHP soit historiquement associé au développement de pages dynamiques côté serveur, son rôle est ici réduit à celui d'un simple distributeur de fichiers. Ainsi, aucune ligne de code PHP n'est exécutée, l'intégralité du traitement reste à la charge du JavaScript exécuté dans le navigateur.

6.4 Architecture générale de l'application

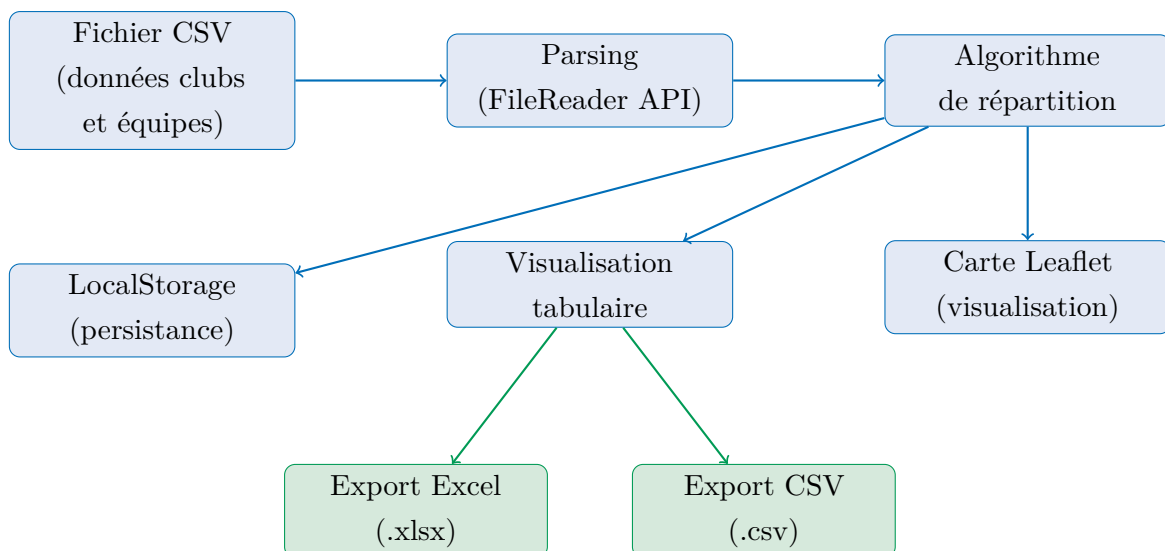


Figure 6.1 – Architecture générale du flux de données de l'application, avec les deux formats d'export (Excel et CSV) accessibles depuis la visualisation tabulaire

Chapitre 7

Implémentation et défis techniques

7.1 Configuration des paramètres de génération

La première étape de l'application consiste à collecter les paramètres de génération via un formulaire accessible sur la page `poule.html`. Ces paramètres définissent le contexte du championnat et orientent l'ensemble des traitements ultérieurs.

Paramètres globaux

L'utilisateur renseigne les informations suivantes :

- **Catégorie** : une chaîne de caractères identifiant la catégorie d'âge (ex. *senior*, *junior*).
- **Genre** : masculin ou féminin, utilisé comme clé de persistance des données.
- **Nombre de niveaux** : un entier compris entre 1 et 5, déterminant combien de niveaux de compétition seront traités séquentiellement.
- **Mode de génération** : deux options sont disponibles.
 - *Distance* : les poules sont formées de manière à minimiser les distances géographiques entre clubs.
 - *Niveau* : les équipes sont réparties selon leur statut sportif (descendant -, stable =, montant +), en respectant des quotas par statut dans chaque poule.
- **Type de distance** :
 - *Vol d'oiseau* : distance calculée par la formule de Haversine.
 - *Voiture* : distance routière obtenue via l'API de navigation de l'IGN (`data.geopf.fr`).

Initialisation du Championnat

Importez vos clubs et définissez les paramètres de base.

Choisissez une catégorie :

Veuillez choisir une option

Nombre de niveaux

1

Type de Championnat

☒ Masculin

☐ Féminin

Critère de génération des poules

☒ Distance

☐ Niveau

Type de distance

☐ À vol d'oiseau

☒ En voiture

Calcul long : les distances routières sont obtenues via l'API IGN avec une limite de 5 requêtes/seconde. Comptez environ **30 secondes par tranche de 10 clubs** dans un même niveau. Le calcul est effectué au moment de la génération des poules de chaque niveau.

Liste des clubs (Format CSV)

Glissez votre fichier ici ou [parcourez](#)

Format .csv uniquement

Le fichier CSV doit inclure les colonnes `latitude` et `longitude` pour chaque club. Vous pouvez géocoder votre fichier au préalable via <https://adresse.data.gouv.fr/csv> target="_blank" rel="noopener" >adresse.data.gouv.fr.

Passer à la configuration des niveaux →

Figure 7.1 – Formulaire de configuration

Paramètres par niveau

Pour chaque niveau traité sur la page `niveau.html`, l'utilisateur précise :

- Le **nombre de poules** souhaité.
- Le **nombre maximum d'équipes par poule**, compris entre 3 et 40.

Avant de lancer la génération, ces valeurs font l’objet de validations croisées :

- La capacité totale doit être suffisante : $\text{nb_poules} \times \text{max_equipes} \geq \text{total_equipes}$.
- La répartition ne doit pas être trop lâche : $\text{total_equipes} > \text{nb_poules} \times (\text{max_equipes} - 1)$.
- Aucun club ne doit avoir plus d’équipes que le nombre de poules (contrainte d’unicité des clubs).

En cas d’infraction, un message d’erreur est affiché et la génération est bloquée.

The screenshot shows a mobile application interface for configuring a tournament. At the top, it says 'SENIOR-MASCULIN-1' and 'Niveau 1 / 3'. Below this, there's a section for 'Équipes (CSV : nom_equipe,nom_club)' with a dashed box containing a file icon and the text '✓ equipes_niveau2.csv'. Underneath are two input fields: 'Nb. poules' and 'Max / poule'. A large blue button labeled 'Générer les poules' is below these. At the bottom, there are two buttons: '← Configuration' and 'Suivant →'. A yellow warning box at the very bottom says '2 Clubs sans coordonnées' and lists 'PONTs DE CE BASKET' and 'SEVRE BASKET A.S.'.

Figure 7.2 – Paramètres par niveau (nombre de poules, max équipes) et avertissement en cas de présence de clubs sans coordonnées fournies à la configuration globale

7.2 Lecture et parsing du fichier CSV contenant les informations sur les clubs

Le référentiel des clubs est fourni sous forme de fichier CSV, traité dans le module `config.js` par la fonction `traiter_csv_clubs`. Ce fichier, issu de la FFBB, liste l’ensemble des clubs avec leurs informations géographiques.

Format attendu

Le format attendu du fichier des clubs est précisé dans la section 3.2.1.

Logique de parsing

La lecture repose sur la méthode `FileReader` de l'API Web. Chaque ligne est découpée via une expression régulière gérant correctement les champs entre guillemets (valeurs contenant des virgules ou des sauts de ligne).

Pour chaque entrée :

1. Les champs identifiant, nom, adresse, commune, code postal, type et coordonnées sont extraits.
2. La validité des coordonnées est vérifiée : latitude et longitude doivent être des nombres flottants (appel à `parseFloat`).
3. Les clubs sans coordonnées valides sont ignorés et consignés dans une liste `clubs_ignorés`, signalée à l'utilisateur via un toast d'avertissement.
4. Les clubs valides sont stockés dans une `Map` indexée par identifiant, facilitant les recherches ultérieures lors du parsing des équipes.

Le résultat est persisté dans le `localStorage` sous la clé `clubs` pour être réutilisé sur les pages suivantes sans nouvel upload.

7.3 Lecture et parsing du fichier CSV contenant les informations sur les équipes à chaque niveau

Pour chaque niveau de compétition, l'utilisateur importe un fichier CSV d'équipes. Le traitement est réalisé par la fonction `traiterCSV` dans `calcul-poules.js`.

Format attendu

Le format attendu du fichier des équipes est précisé dans la section [3.2.2](#).

Logique de parsing et validations

1. **Vérification des en-têtes** : le parseur vérifie que toutes les colonnes obligatoires sont présentes. En cas d'en-tête manquant, une erreur bloquante est remontée.
2. **Résolution du club** : chaque ligne est croisée avec la `Map` des clubs ; si l'identifiant `CLUB_NUMERO` est inconnu, l'équipe est mise de côté et listée dans un rapport d'équipes non reconnues.
3. **Cohérence des données** : si deux lignes portent le même `CLUB_NUMERO` mais des noms différents, une alerte est émise.
4. **Gestion des doublons** : les numéros d'équipes en doublon sont incrémentés automatiquement pour garantir l'unicité des identifiants.

5. **Construction de l'objet équipe** : les coordonnées géographiques sont récupérées depuis le référentiel clubs et associées à chaque équipe.

7.4 Génération des poules

Le cœur de l'application réside dans les algorithmes de génération de poules, déclinés en deux variantes selon le mode choisi.

7.4.1 Mode distance (`calcul-poules.js`)

L'objectif est de former des poules géographiquement cohérentes, c'est-à-dire où les clubs d'une même poule sont les plus proches possible les uns des autres.

Calcul des distances

Deux métriques de distance sont supportées selon le paramètre choisi lors de la configuration :

- **Vol d'oiseau** : la distance entre deux équipes est calculée à la volée à chaque évaluation, directement via la formule de Haversine [7] :

Formule de Haversine

$$d = 2R \cdot \arctan 2\left(\sqrt{a}, \sqrt{1-a}\right), \quad a = \sin^2 \frac{\Delta\phi}{2} + \cos \phi_1 \cos \phi_2 \sin^2 \frac{\Delta\lambda}{2}$$

avec $R = 6\,371$ km.

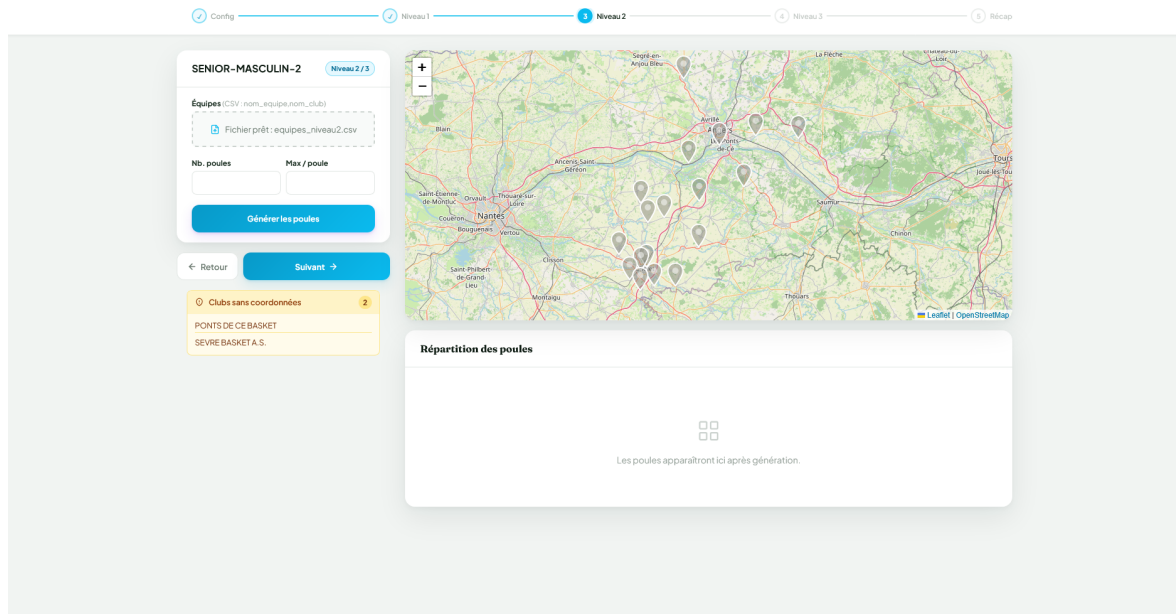
Aucune structure de cache n'est nécessaire dans ce cas : le calcul est suffisamment rapide pour être effectué à la demande.

- **Distance routière** : les distances sont obtenues via l'API IGN (data.geopf.fr/navigation/itineraire) [8] Le coût en temps réseau étant significatif (environ 30 secondes pour 10 clubs), une matrice symétrique est précalculée pour toutes les paires de clubs du niveau avant la génération. Les requêtes sont limitées à 4 par seconde pour respecter les quotas de l'API. La matrice est ensuite mise en cache dans le `localStorage` afin d'éviter tout recalcul lors des rechargements de page.

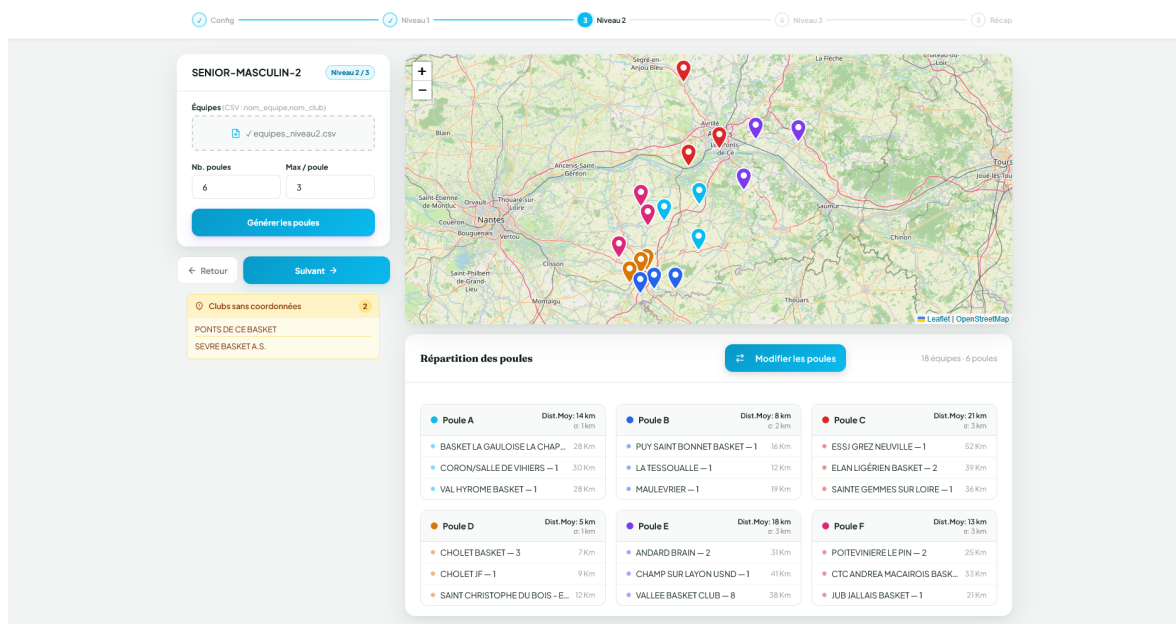
Déroulement de l'algorithme

L'algorithme, décrit en détail au chapitre 4.2, procède en plusieurs phases successives : initialisation des poules par sélection de graines géographiquement dispersées, affectation gloutonne de chaque équipe à la poule dont le barycentre est le plus proche, mécanisme

de sauvetage pour les équipes non plaçables, puis optimisation itérative par échanges entre poules pour minimiser la variance des distances moyennes. La contrainte d'unicité de club est vérifiée à chaque étape.



(a) Avant la génération : formulaire de paramétrage (nombre de poules, max équipes) et carte en attente d'équipes.



(b) Après la génération : carte Leaflet avec les marqueurs colorés par poule et grille des cartes de poules avec leurs statistiques.

Figure 7.3 – Page de génération en mode distance, avant et après exécution de l'algorithme.

7.4.2 Mode niveau (calcul-poules-niveau.js)

Ce mode vise à équilibrer les poules selon le statut sportif des équipes (descendant -, stable =, montant +), tout en minimisant secondairement les distances géographiques. Comme détaillé au chapitre 4.3, des quotas par statut sont calculés pour chaque poule afin de garantir une répartition homogène, puis les équipes sont distribuées statut par statut avec un mécanisme de sauvetage en cas de blocage et une phase d'optimisation géographique par échanges, sous contrainte de ne pas altérer l'équilibre des statuts.

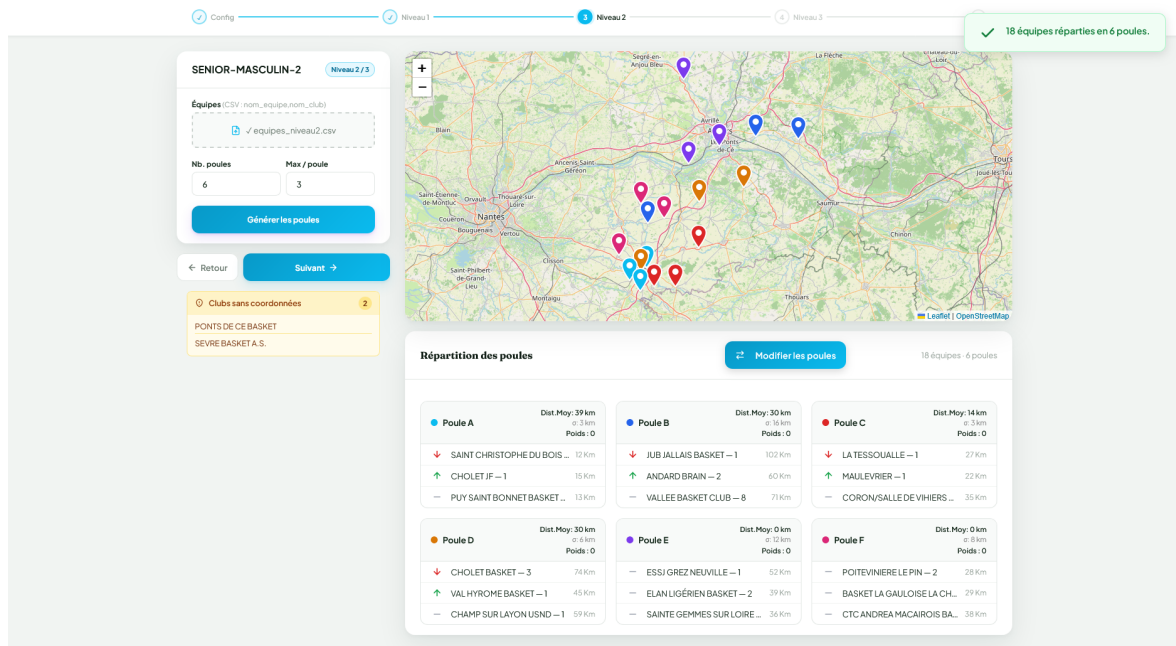


Figure 7.4 – Résultat de la génération en mode niveau : les cartes de poules affichent les flèches directionnelles (+/=/-) indiquant le statut de chaque équipe et le poids global de la poule.

7.5 Visualisation cartographique avec Leaflet

La carte interactive est rendue via la bibliothèque **Leaflet.js** sur un fond de carte OpenStreetMap.

Initialisation

```
const map = L.map("map").setView([46.6033, 1.8883], 6);
L.tileLayer("https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png")
  .addTo(map);
```

La vue est centrée sur la France métropolitaine au niveau de zoom 6.

Marqueurs et couleurs

Chaque équipe est représentée par un marqueur SVG personnalisé. Une palette de 10 couleurs distinctes est attribuée cycliquement aux poules. Lorsque plusieurs équipes sont géolocalisées au même endroit (gymnase partagé), leurs marqueurs sont disposés en spirale autour du point d'origine avec un rayon de 500 m pour rester lisibles :

$$(\phi', \lambda') = \left(\phi + r \sin\left(\frac{2\pi k}{n}\right), \lambda + r \cos\left(\frac{2\pi k}{n}\right) \right)$$

avec $r = 0,005$ et k l'indice de l'équipe parmi les n colocalisées.

Figure 7.5 – Vue générale de la carte Leaflet : chaque marqueur SVG est coloré selon sa poule d'appartenance ; les équipes colocalisées sont disposées en spirale pour éviter les superpositions.

Interaction

- **Survol** : une infobulle affiche le nom et le numéro de l'équipe ainsi que l'identifiant du club.
- **Mise en évidence par poule** : un clic sur la carte d'une poule dans le tableau rend ses marqueurs opaques (opacité 1,0) et recadre la vue sur ses équipes, tandis que les marqueurs des autres poules sont atténués (opacité 0,2).
- **Mode édition** : les marqueurs deviennent cliquables pour initier un échange d'équipes directement depuis la carte.

Figure 7.6 – Infobulle au survol d'un marqueur : nom, numéro et identifiant du club.

Figure 7.7 – Mise en évidence d'une poule : ses marqueurs sont opaques, les autres atténués, et la vue est recadrée.

7.6 Visualisation tabulaire

En parallèle de la carte, les poules sont affichées sous forme de cartes (*cards*) dans une grille CSS responsive. Chaque carte présente :

- Un **en-tête** avec le nom de la poule (Poule A, B, ...), un indicateur de couleur et un lien « voir sur carte ».
- Des **statistiques** : distance moyenne intra-poule (km), écart-type des distances au barycentre, et poids en mode niveau.
- La **liste des équipes** avec, pour chacune, un indicateur visuel (point coloré en mode distance, flèche directionnelle en mode niveau), le nom et numéro de l'équipe, et sa distance totale aux autres membres de la poule.

- Une ligne *Exempt* grisée si la poule dispose de places libres.

Figure 7.8 – Grille de cartes de poules en mode distance : chaque carte affiche son en-tête coloré, les statistiques (distance moyenne, écart-type) et la liste des équipes avec leur distance totale. La ligne *Exempt* apparaît en gris si la poule a une place libre.

Calcul des statistiques affichées

- **Distance moyenne** $\bar{d}$: moyenne de toutes les distances inter-équipes de la poule ($\frac{n(n-1)}{2}$ paires).
- **Écart-type** σ : racine carrée de la variance des distances au barycentre.
- **Poids** (mode niveau) : somme algébrique des statuts (+1 pour +, 0 pour =, -1 pour -).

7.6.1 Possibilité d'échange manuel des équipes

L'utilisateur peut activer le **mode édition** via le bouton « *Modifier les poules* ». Ce mode permet de réorganiser manuellement les équipes entre poules par double clic successif.

Mécanisme de sélection et d'échange

1. **Premier clic** : l'équipe sélectionnée est mise en évidence (classe CSS `selected`).
Le système mémorise l'indice de poule et l'identifiant d'équipe.
2. **Deuxième clic** : le système tente l'échange entre les deux équipes sélectionnées.

Figure 7.9 – Mode édition activé : le badge « *Mode édition* » est visible, le message d'aide s'affiche, et une équipe est mise en surbrillance (fond ambré) après le premier clic.

Contraintes vérifiées

Avant d'appliquer l'échange, plusieurs conditions sont contrôlées :

- **Unicité de club** : l'échange ne doit pas conduire à la présence de deux équipes du même club dans une même poule.
- **Gestion des exempts** : l'échange est interdit s'il conduit à la présence de deux exempts dans une même poule. Ce cas peut notamment se produire lors d'un échange entre une équipe appartenant à une poule contenant déjà un exempt et une position *Exempt*.

En cas de violation, une animation de secousse (*card-shake*) est appliquée à la carte concernée et un toast explicatif est affiché.

Figure 7.10 – Échange validé : toast de confirmation « *Échange effectué* » et mise à jour immédiate des statistiques.

Figure 7.11 – Violation de la contrainte d’unicité : animation de secousse sur la carte et toast d’erreur explicatif.

Mise à jour des statistiques

Après chaque échange validé, les barycentres, les distances parcourues par toutes les équipes des poules affectées, ainsi que les distances moyennes et les écarts-types sont recalculés à la volée. La carte et le tableau se mettent à jour immédiatement sans rechargement de page.

7.7 Persistance des données avec LocalStorage

Toutes les données de session sont persistées dans le `localStorage` du navigateur, permettant de naviguer entre les pages sans perte d’information et de reprendre le travail après un rechargement.

Clés de stockage

Clé	Contenu
<code>championnatConfig</code>	Configuration globale (catégorie, genre, niveaux, mode...)
<code>clubs</code>	Tableau de tous les clubs valides
<code>clubs_ignorés</code>	Liste des clubs sans coordonnées
<code>csv_equipes_niveauN</code>	Équipes du niveau N
<code>csv_nom_niveauN</code>	Nom du fichier CSV du niveau N
<code>{categ}-{genre}-N</code>	Poules générées pour le niveau N
<code>matriceDistances:niveauN</code>	Matrice des distances pour le niveau N

Figure 7.12 – Contenu du `localStorage` dans les outils de développement du navigateur : les clés de configuration, clubs, équipes et matrices de distances sont visibles après une session complète.

Points de sauvegarde

- **Upload clubs** : les clubs valides et ignorés sont sauvegardés immédiatement après parsing.

- **Upload équipes** : le tableau d'équipes et le nom de fichier sont sauvegardés par niveau.
- **Navigation vers le niveau suivant** : les poules du niveau courant sont sauvegardées avant la redirection, garantissant que l'utilisateur peut revenir en arrière sans perdre son travail.
- **Calcul de la matrice distances** : la matrice est persistée dès sa construction complète pour éviter de rappeler l'API IGN lors des rechargements.

Restauration au chargement

À l'ouverture de `niveau.html`, l'application :

1. Charge la configuration depuis `championnatConfig`; en l'absence de cette clé, l'utilisateur est redirigé vers la page de configuration.
2. Restaure les équipes du niveau courant si elles ont déjà été importées.
3. Restaure les poules précédemment générées, pré-remplissant automatiquement le formulaire et réaffichant la carte et le tableau.

7.8 Affichage des résultats et export

L'application propose plusieurs interfaces de visualisation permettant à l'organisateur de consulter les résultats générés à différentes étapes du processus. Ces vues offrent également des fonctionnalités d'export afin de faciliter l'exploitation et l'archivage des données.

7.8.1 Récapitulatif des poules

La page `recapitulatif.html` présente une vue synthétique de toutes les poules générées pour l'ensemble des niveaux du championnat. Elle est alimentée par les données persistées dans le `localStorage`.

Pour chaque niveau, les poules sont affichées sous le même format tabulaire que sur la page de génération. Une vue consolidée indique le nombre total d'équipes, le nombre de poules et la distance moyenne globale.

Figure 7.13 – Page de récapitulatif affichant l'ensemble des poules de tous les niveaux du championnat, avec les statistiques consolidées et les boutons d'export.

Deux formats d'export sont proposés :

- **Excel (.xlsx)** : via la bibliothèque XLSX.js. Le classeur contient une feuille par niveau. Chaque poule est représentée par un bloc comprenant un en-tête avec les statistiques, puis la liste des équipes avec leur numéro de club et leur distance totale.
- **CSV** : format tabulaire plat listant les équipes, niveaux et poules.

Figure 7.14 – Fichier Excel exporté ouvert dans un tableur : une feuille par niveau, chaque poule présentée en bloc avec ses statistiques et la liste de ses équipes.

7.8.2 Résultat de la numérotation

À l'issue de la numérotation, le résultat est affiché dans l'interface sous forme de tableau par poule et par niveau. Pour chaque équipe sont indiqués le numéro attribué, le nom de l'équipe, le club ainsi que le souhait associé. Les lignes *Exempt* sont affichées avec un style distinct afin d'être facilement identifiables.

L'organisateur peut également exporter le résultat de la numérotation dans deux formats :

- **CSV** : même structure que le fichier d'entrée avec une colonne **Numéro** ajoutée, encodé en UTF-8 avec BOM afin d'assurer une compatibilité optimale avec Excel.
- **Excel (.xlsx)** : fichier généré côté navigateur via la bibliothèque SheetJS **sheetjs**, contenant une feuille unique avec des largeurs de colonnes adaptées automatiquement au contenu.

Chapitre 8

Conclusion et perspectives

8.1 Bilan du projet

8.2 Perspectives d'évolution

En conclusion, ce projet m'a permis de ...

Bibliographie et Webographie

Articles et ressources algorithmiques

- [1] S. P. LLOYD, “Least Squares Quantization in PCM,” *IEEE Transactions on Information Theory*, t. 28, n° 2, p. 129-137, 1982.
- [2] D. ARTHUR et S. VASSILVITSKII, “k-means++ : The Advantages of Careful Seeding,” in *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 2007, p. 1027-1035.

Outils utilisés

- [3] LEAFLET CONTRIBUTORS, *Leaflet — a JavaScript library for interactive maps*, 2024. visité le 8 mai 2026. adresse : <https://leafletjs.com/>.
- [4] GOOGLE, *Google Maps Platform — Documentation*, 2024. visité le 8 mai 2026. adresse : <https://developers.google.com/maps>.
- [5] OPENLAYERS CONTRIBUTORS, *OpenLayers — A high-performance mapping library*, 2024. visité le 8 mai 2026. adresse : <https://openlayers.org/>.
- [6] MAPBOX, *Mapbox GL JS*, 2024. visité le 8 mai 2026. adresse : <https://www.mapbox.com/>.
- [8] INSTITUT NATIONAL DE L’INFORMATION GÉOGRAPHIQUE ET FORESTIÈRE (IGN), *API Géoplateforme — Calcul d’itinéraire*, Documentation officielle, Géoplateforme, Endpoint : <https://data.geopf.fr/navigation/itineraire>, limite : 5 requêtes/seconde par IP, 2024. visité le 8 mai 2026. adresse : <https://cartes.gouv.fr/aide/fr/guides-utilisateur/utiliser-les-services-de-la-geoplateforme/calcul-itineraire/>.

Documentation officielle

- MDN Web Docs – *FileReader API*. <https://developer.mozilla.org/fr/docs/Web/API/FileReader> [Consulté le JJ/MM/2026]

- MDN Web Docs – *Window.localStorage*. <https://developer.mozilla.org/fr/docs/Web/API/Window/localStorage> [Consulté le JJ/MM/2026]
- Leaflet.js – *Documentation officielle*. <https://leafletjs.com/reference.html> [Consulté le JJ/MM/2026]
- OpenStreetMap – *Tile Usage Policy*. <https://operations.osmfoundation.org/policies/tiles/> [Consulté le JJ/MM/2026]
- *Visual Studio Code* (éditeur de code). <https://code.visualstudio.com>
- *Git & GitHub* (gestion de versions). <https://github.com>

Articles et ressources algorithmiques

- SINNOTT, R. W. (1984). *Virtues of the Haversine*. *Sky and Telescope*, 68(2), 158.
- Wikipedia – *Algorithme des k-moyennes (K-means)*. <https://fr.wikipedia.org/wiki/K-moyennes> [Consulté le JJ/MM/2026]

Glossaire

API	<i>(Application Programming Interface)</i> – Interface de programmation permettant à deux logiciels de communiquer entre eux.
Barycentre	En géographie, point moyen d'un ensemble de coordonnées GPS, utilisé pour représenter le centre géographique d'un groupe d'équipes.
CSV	<i>(Comma-Separated Values)</i> – Format de fichier texte simple dans lequel les données sont séparées par des virgules.
DOM	<i>(Document Object Model)</i> – Représentation arborescente d'une page HTML manipulable par JavaScript.
FFBB	<i>(Fédération Française de BasketBall)</i> – organisme chargé de l'organisation et de la gestion des compétitions de basket-ball en France.
FileReader	API Web native de JavaScript permettant de lire le contenu de fichiers locaux sélectionnés par l'utilisateur.
Haversine	Formule trigonométrique permettant de calculer la distance entre deux points à la surface d'une sphère à partir de leurs coordonnées géographiques.
K-means	Algorithme de partitionnement qui regroupe des points en k clusters en minimisant la distance intra-cluster.
Leaflet	Bibliothèque JavaScript open source légère pour la création de cartes interactives dans le navigateur.
LocalStorage	API Web permettant de stocker des données clé-valeur de façon persistante dans le navigateur de l'utilisateur.
Poule	Groupe d'équipes sportives de même niveau amenées à se rencontrer lors d'une phase de compétition.
Stepper	Composant d'interface utilisateur guidant l'utilisateur à travers un processus séquentiel en plusieurs étapes distinctes.
WGS84	Système géodésique de référence mondial utilisé par le GPS, exprimant les positions en latitude/longitude décimaux.